

Equações Diferenciais Numéricas

Trabalho Prático 3: A Equação do Calor

Mateus Ryan de Castro Lima
UFMG

 github.com/Mryan31/tp3_EDN

Novembro de 2025

Sumário

1	Introdução	3
2	Séries de Fourier	3
2.1	Implementação Computacional	3
2.2	Estimativa do Número de Termos N	4
3	Método das Linhas	5
3.1	Implementação do MOL	5
3.2	Análise de Convergência	6
4	Métodos de Diferenças Finitas	6
4.1	FTCS (Explícito / Progressivo)	7
4.2	BTCS (Implícito / Regressivo)	7
4.3	Crank-Nicholson	8
5	Conclusão	9

1 Introdução

Este trabalho investiga diferentes métodos numéricos para resolver a equação do calor unidimensional

$$\begin{cases} \partial_t u = \Delta u, & t > 0, x \in (0, 1), \\ u(0, t) = u(1, t) = 0, \\ u(x, 0) = f(x), \end{cases}$$

onde a condição inicial é a função “tenda” (ou onda triangular) definida por

$$f(x) = \begin{cases} 2x, & 0 < x \leq 1/2, \\ 2 - 2x, & 1/2 < x < 1. \end{cases}$$

Os métodos abordados incluem:

1. **Séries de Fourier:** Utilizada como solução de referência (analítica).
2. **Método das Linhas (MOL):** Discretização espacial mantendo o tempo contínuo.
3. **Diferenças Finitas:** Métodos totalmente discretos.

O objetivo principal é analisar a convergência e a estabilidade dessas abordagens, com foco especial na influência do parâmetro de estabilidade μ . Todos os experimentos utilizam um tempo final $T = 0.5$.

2 Séries de Fourier

A solução exata para este problema com condições de fronteira de Dirichlet homogêneas pode ser escrita como uma série infinita:

$$u(x, t) = \sum_{n=1}^{\infty} b_n \sin(n\pi x) e^{-(n\pi)^2 t},$$

onde os coeficientes b_n são dados pela projeção da condição inicial:

$$b_n = 2 \int_0^1 f(x) \sin(n\pi x) dx.$$

Para a função $f(x)$ considerada, a integração resulta em:

$$b_n = \frac{8 \sin(n\pi/2)}{(n\pi)^2}.$$

Note que para n par, $b_n = 0$.

2.1 Implementação Computacional

A implementação da solução de Fourier vetorizada é apresentada abaixo.

Listing 1: Implementação da Solução via Fourier

```

1 def f_x(x):
2     """Condicao inicial f(x) (Onda Triangular)."""
3     return np.piecewise(x,
4         [np.logical_and(x > 0, x <= 0.5),
5          np.logical_and(x > 0.5, x < 1)],
6         [lambda x: 2*x,
7          lambda x: 2 - 2*x])
8
9 def get_bn(n):
10    """Calcula coeficiente b_n."""
11    if n % 2 == 0:
12        return 0.0
13    n_pi = n * np.pi
14    return (8 * np.sin(n_pi / 2)) / (n_pi**2)
15
16 def solucao_fourier(x_vec, t, N):
17    """Calcula u(x,t) truncando a serie em N termos."""
18    if isinstance(t, np.ndarray):
19        x_grid, t_grid = np.meshgrid(x_vec, t)
20    else:
21        x_grid, t_grid = x_vec, t
22
23    U = np.zeros_like(x_grid, dtype=float)
24    for n in range(1, N + 1):
25        bn = get_bn(n)
26        if bn != 0.0:
27            termo = bn * np.sin(n * np.pi * x_grid) * np.exp(-(n * np.pi
28                )**2 * t_grid)
29            U += termo
30    return U

```

2.2 Estimativa do Número de Termos N

Para garantir que o erro de truncamento seja inferior a $\epsilon = 0.01$, analisamos a cauda da série. Sabendo que $|\sin(\cdot)e^{-\cdot}| \leq 1$, temos:

$$|u - u_N| \leq \sum_{n=N+1}^{\infty} |b_n| \approx \frac{8}{\pi^2} \sum_{n=N+1}^{\infty} \frac{1}{n^2} \approx \frac{8}{\pi^2 N}.$$

Impondo $\frac{8}{\pi^2 N} \leq 0.01$, obtemos $N \geq \frac{800}{\pi^2} \approx 81$. O código utiliza uma margem de segurança, adotando `**N = 86**` para todas as comparações.

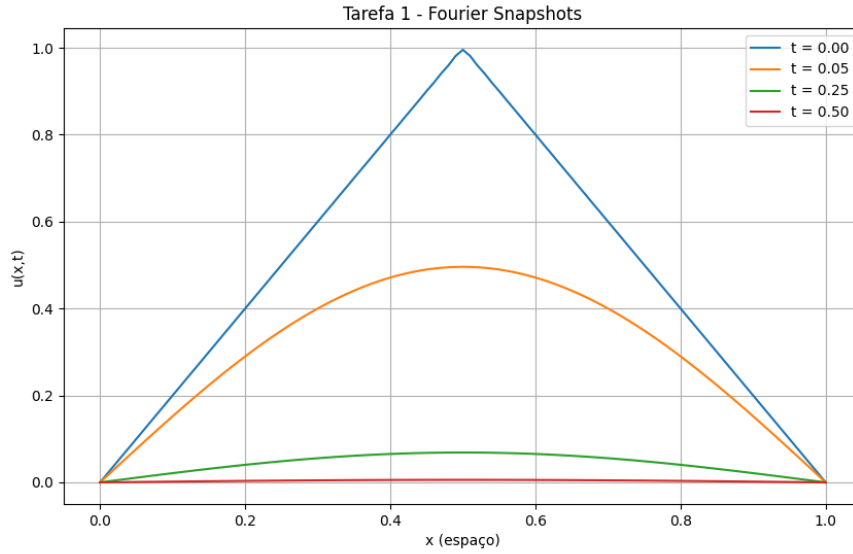


Figura 1: Solução de referência obtida via séries de Fourier com $N = 86$.

3 Método das Linhas

O Método das Linhas (Method of Lines - MOL) discretiza o domínio espacial $x_j = j\Delta x$ mas mantém a variável temporal contínua. Aproximando a derivada espacial por diferenças finitas centradas, a EDP é convertida em um sistema de Equações Diferenciais Ordinárias (EDOs):

$$\frac{dU_j}{dt} = \frac{U_{j+1} - 2U_j + U_{j-1}}{(\Delta x)^2}.$$

Este sistema foi resolvido utilizando o integrador `solve_ivp` do SciPy (método RK45).

3.1 Implementação do MOL

Listing 2: Sistema de EDOs para o MOL

```

1 def sistema_MOL(t, U, M, dx_quadrado_inv):
2     """Define o sistema de EDOs: dU/dt = A*U."""
3     dUdt = np.zeros_like(U)
4     # Pontos internos
5     for j in range(1, M - 2):
6         dUdt[j] = (U[j+1] - 2*U[j] + U[j-1]) * dx_quadrado_inv
7
8     # Fronteiras (U_0=0, U_M=0)
9     dUdt[0] = (U[1] - 2*U[0] + 0.0) * dx_quadrado_inv
10    dUdt[-1] = (0.0 - 2*U[-1] + U[-2]) * dx_quadrado_inv
11    return dUdt
12
13 def solucao_MOL(delta_x, t_final, t_pontos):
14     L = 1.0
15     M = int(L/delta_x)+1
16     x_vec = np.linspace(0, L, M)
17     x_internos = x_vec[1:-1]
```

```

18
19 # Resolve o PVI
20 sol = solve_ivp(
21     fun=sistema_MOL, t_span=[0,t_final],
22     y0=f_x(x_internos), t_eval=t_pontos,
23     args=(len(x_internos), 1/(delta_x**2))
24 )
25
26 # Remonta a solucao completa com as bordas
27 U_completo = np.zeros((M,len(t_pontos)))
28 U_completo[1:-1,:] = sol.y
29 return x_vec, sol.t, U_completo

```

3.2 Análise de Convergência

Foram testados os espaçamentos $\Delta x \in \{0.1, 0.05, 0.025, 0.0125\}$. A Figura 2 apresenta o erro L2 no tempo final. Observa-se que o erro decai quadraticamente, confirmando a ordem $\mathcal{O}(\Delta x^2)$ da aproximação espacial centrada.

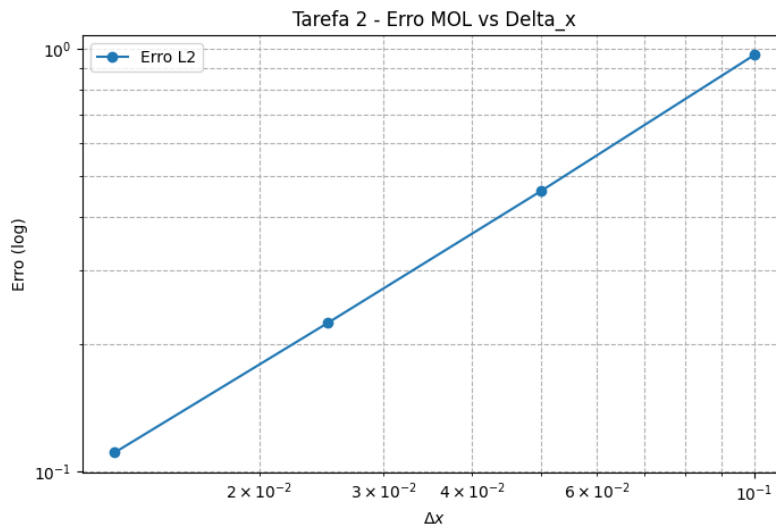


Figura 2: Erro do Método das Linhas em função de Δx (escala log-log).

4 Métodos de Diferenças Finitas

Nesta seção, o tempo também é discretizado. A estabilidade numérica depende do parâmetro μ (ou número de Courant difusivo r):

$$\mu = \frac{\Delta t}{(\Delta x)^2}.$$

Conforme solicitado, foram testados os valores $\mu_1 = 5/11 \approx 0.45$ e $\mu_2 = 5/9 \approx 0.55$, utilizando uma malha fixa com $\Delta x = 0.05$ ($M = 21$ pontos).

O enunciado refere-se aos métodos como "aproximações progressivas" e "regressivas". No contexto numérico da equação do calor:

- **Aproximação Progressiva** refere-se à diferença progressiva no tempo, resultando no esquema explícito **FTCS** (Forward-Time Central-Space).
- **Aproximação Regressiva** refere-se à diferença regressiva no tempo, resultando no esquema implícito **BTCS** (Backward-Time Central-Space).

4.1 FTCS (Explícito / Progressivo)

O esquema explícito é dado por $U_j^{n+1} = U_j^n + \mu(U_{j+1}^n - 2U_j^n + U_{j-1}^n)$. Ele é condicionalmente estável, exigindo $\mu \leq 0.5$.

Listing 3: Solver FTCS

```

1 def solver_ftcs(U, M, N, r):
2     for n in range(N):
3         for j in range(1, M - 1):
4             U[j, n+1] = U[j, n] + r * (U[j+1, n] - 2*U[j, n] + U[j-1, n])
5     return U

```

Como $\mu_2 = 0.55 > 0.5$, espera-se instabilidade, o que é confirmado pelos resultados gráficos na Figura 3.

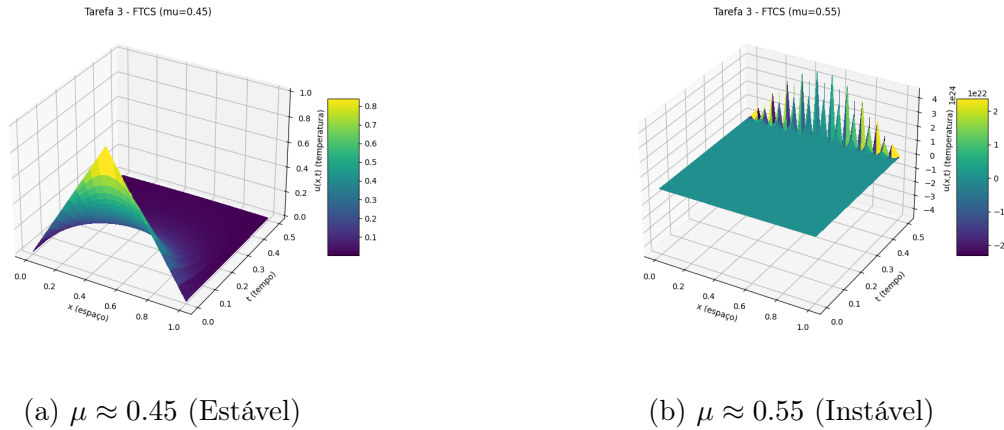


Figura 3: Comportamento do método FTCS. Nota-se a explosão numérica para $\mu > 0.5$.

4.2 BTCS (Implícito / Regressivo)

O esquema implícito resolve um sistema linear tridiagonal a cada passo. É incondicionalmente estável para qualquer $\mu > 0$.

Listing 4: Solver BTCS (usando matrizes bandadas)

```

1 def solver_btcs(U, M, N, r):
2     # Configuracao das diagonais para sistema (M-2)x(M-2)
3     diag_sup = np.full(M - 3, -r)
4     diag_main = np.full(M - 2, 1 + 2*r)
5     diag_inf = np.full(M - 3, -r)
6
7     A_banded = np.zeros((3, M - 2))
8     A_banded[0, 1:] = diag_sup
9     A_banded[1, :] = diag_main

```

```

10 A_banded[2, :-1] = diag_inf
11
12 for n in range(N):
13     b = U[1:-1, n]
14     U[1:-1, n+1] = solve_banded((1, 1), A_banded, b)
15 return U

```

A Figura 4 mostra que o método permanece estável mesmo para o caso $\mu = 0.55$ onde o FTCS falhou.

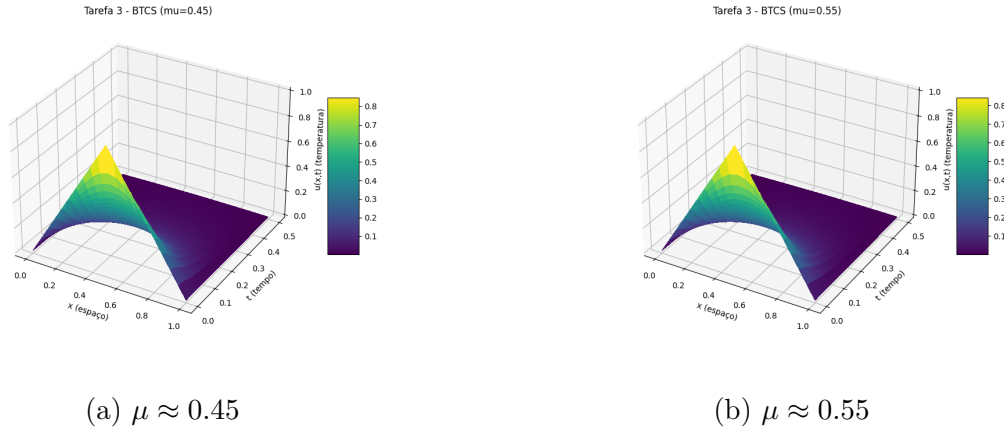


Figura 4: Método BTCS: Estabilidade garantida em ambos os casos.

4.3 Crank-Nicholson

Este método é uma média dos esquemas FTCS e BTCS, sendo também incondicionalmente estável, mas com ordem de convergência $\mathcal{O}(\Delta t^2)$ no tempo, superior aos anteriores.

Listing 5: Solver Crank-Nicholson

```

1 def solver_crank_nicholson(U, M, N, r):
2     r2 = r / 2.0
3     diag_sup_A = np.full(M - 3, -r2)
4     diag_main_A = np.full(M - 2, 1 + r)
5     diag_inf_A = np.full(M - 3, -r2)
6
7     A_banded = np.zeros((3, M - 2))
8     A_banded[0, 1:] = diag_sup_A
9     A_banded[1, :] = diag_main_A
10    A_banded[2, :-1] = diag_inf_A
11
12    for n in range(N):
13        U_n = U[1:-1, n]
14        b = np.zeros(M - 2)
15        # Construção do lado direito (explícito)
16        for j in range(1, M - 3):
17            b[j] = r2 * U_n[j-1] + (1 - r) * U_n[j] + r2 * U_n[j+1]
18        # Tratamento das fronteiras no vetor b...
19        b[0] = r2 * U[0, n] + (1 - r) * U_n[0] + r2 * U_n[1]
20        b[-1] = r2 * U_n[-2] + (1 - r) * U_n[-1] + r2 * U[M-1, n]
21
22        U[1:-1, n+1] = solve_banded((1, 1), A_banded, b)
23    return U

```

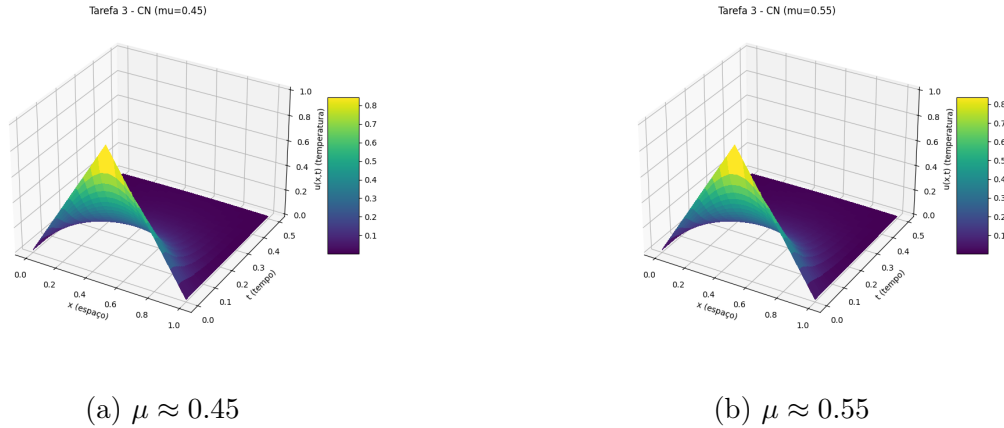



Figura 5: Método Crank-Nicholson: Estável e com maior precisão.

5 Conclusão

A implementação dos métodos numéricos permitiu verificar na prática os conceitos de consistência e estabilidade para a equação do calor.

- O Método das Linhas mostrou-se uma ferramenta poderosa, convertendo a EDP em um sistema rígido de EDOs que foi resolvido eficientemente com passo adaptativo, exibindo a convergência espacial esperada.
- A análise das Diferenças Finitas confirmou o limite de estabilidade do método explícito (FTCS) em $\mu = 0.5$. O teste com $\mu \approx 0.55$ resultou em divergência catastrófica, enquanto os métodos implícitos (BTCS e Crank-Nicholson) permaneceram estáveis.
- Crank-Nicholson destaca-se como a melhor opção geral, pois oferece estabilidade incondicional combinada com precisão de segunda ordem no tempo, ao custo computacional similar ao do BTCS (resolução de sistemas tridiagonais).

Os códigos completos e arquivos auxiliares utilizados para gerar este relatório encontram-se no repositório GitHub indicado.

Referências

- [1] MILANÉS, Aniura. *Equações Diferenciais Numéricas: Notas de Aula*. Departamento de Matemática, UFMG, 2021.
- [2] K. Atkinson, W. Han, and D. E. Stewart. *Numerical solution of ordinary differential equations*. John Wiley & Sons, 2011.
- [3] R. J. LeVeque. *Finite difference methods for ordinary and partial differential equations*. SIAM, 2007.
- [4] K. W. Morton and D. F. Mayers. *Numerical solution of partial differential equations*. Cambridge University Press, 2005.